

Grand Mémo Récapitulatif : HTML5, CSS3 & JavaScript / Java

Guide complet des notions fondamentales, balises, propriétés, structures de données et programmation orientée objet.

1. HTML5 — STRUCTURE, BALISES & SÉMANTIQUE

HTML (HyperText Markup Language) définit le squelette et la structure sémantique d'une page Web.

1.1 Structure de base d'un document HTML5

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Titre de la page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- Contenu visible de la page -->
  <script src="script.js"></script>
</body>
</html>
```

1.2 Les Balises Sémantiques Essentielles

Balise	Rôle & Description
<code><header></code>	En-tête de la page ou d'une section (titres, logo, navigation).
<code><nav></code>	Contient les liens principaux de navigation.
<code><main></code>	Contenu principal et unique du document (exclut headers/footers répétitifs).
<code><section></code>	Regroupe du contenu thématique (ex: "À propos", "Compétences", "Projets").
<code><article></code>	Bloc autonome réutilisable (post de blog, fiche projet, actualité).
<code><aside></code>	Contenu secondaire/annexe (barre latérale, liens connexes).
<code><footer></code>	Pied de page (mentions légales, droits d'auteur, contacts).

1.3 Éléments de Contenu, Textes et Formulaires

- **Titres :** `<h1>` (titre principal) jusqu'à `<h6>`.
- **Conteneurs :** `<div>` (conteneur bloc générique), `<span>` (conteneur en ligne/inline).
- **Listes :** `<ul>` (non ordonnée), `<ol>` (ordonnée), avec des puces `<li>`.
- **Liens & Médias :** `<a href="URL">`, ``.
- **Formulaires :**

```
<form action="#" method="POST">
  <label for="nom">Nom :</label>
  <input type="text" id="nom" name="nom" required placeholder="Votre nom">
```

```
<label for="email">Email :</label>
<input type="email" id="email" name="email" required>

<button type="submit">Envoyer</button>
</form>
```

2. CSS3 — MARGE, STYLES, LAYOUTS & RESPONSIVITÉ

CSS (Cascading Style Sheets) gère la mise en forme visuelle, la mise en page et l'adaptabilité aux écrans (Responsive Design).

2.1 Modèle de Boîte (Box Model)

Chaque élément HTML est représenté sous forme de boîte rectangulaire composée de :

- **Content** : Le contenu texte/image réel.
- **Padding** : Marge interne (espace entre le contenu et la bordure).
- **Border** : La bordure entourant le padding.
- **Margin** : Marge externe (espace séparant l'élément des éléments voisins).

```
/* Règle d'or absolue pour éviter les dépassements de taille */
*, *::before, *::after {
  box-sizing: border-box; /* Inclusion de padding et border dans la largeur totale */
  margin: 0;
  padding: 0;
}
```

2.2 Positionnement & Layouts Moderne (Flexbox & Grid)

Propriété	Valeurs Courantes & Rôle
<code>position</code>	<code>static</code> <code>relative</code> <code>absolute</code> <code>fixed</code> <code>sticky</code>
<code>display: flex</code>	Disposition unidirectionnelle (ligne ou colonne). <code>flex-direction: row column;</code> <code>justify-content: center space-between flex-start;</code> (axe principal) <code>align-items: center stretch;</code> (axe secondaire)
<code>display: grid</code>	Disposition bidimensionnelle (lignes + colonnes). <code>grid-template-columns: repeat(3, 1fr);</code> <code>gap: 20px;</code> (espacement entre cellules)

2.3 Sélecteurs & Media Queries (Responsive)

```
/* Sélecteurs */
p { color: #ffffff; } /* Par balise */
.carte { background: #1e293b; } /* Par classe */
#header { padding: 10px; } /* Par ID */
.carte:hover { transform: scale(1.05); } /* Pseudo-classe */

/* Media Query : Adaptation aux mobiles */
@media (max-width: 768px) {
  .menu {
    flex-direction: column;
  }
}
```

3. JAVASCRIPT & JAVA — BASES & FONCTIONNALITÉS

Bien que leurs noms se ressemblent, **JavaScript** est le langage de script dynamique du Web (côté navigateur / Node.js), tandis que **Java** est un langage compilé orienté objet (exécuté sur la JVM).

3.1 Comparatif des Concepts Fondamentaux

Concept	JavaScript (Web / Dynamique)	Java (Compilé / Typé)
Variables	<code>let x = 10;</code> (réassignable) <code>const y = "Hello";</code> (constante)	<code>int x = 10;</code> <code>String y = "Hello";</code> (Typage statique)
Affichage	<code>console.log("Texte");</code>	<code>System.out.println("Texte");</code>
Fonctions / Méthodes	<code>function nom(p) { return p; }</code> <code>const nom = (p) => p;</code>	<code>public int nom(int p) { return p; }</code>
Tableaux / Arrays	<code>const arr = [1, 2, 3];</code> <code>arr.push(4);</code>	<code>int[] arr = {1, 2, 3};</code> <code>ArrayList<Integer> list = new ArrayList<>();</code>

3.2 Contrôle de Flux (Conditions & Boucles)

Les structures de contrôle sont très similaires dans les deux langages :

```
// Condition IF / ELSE
if (age >= 18) {
    // Majeur
} else {
    // Mineur
}

// Boucle FOR (Parcours classique)
for (let i = 0; i < 10; i++) {
    // Répéter 10 fois
}

// Boucle WHILE
while (condition) {
    // Exécution tant que la condition est vraie
}
```

3.3 JavaScript : Manipulation du DOM (Document Object Model)

JavaScript permet de rendre une page HTML interactive en modifiant directement ses éléments :

```
// 1. Sélectionner un élément HTML
const bouton = document.querySelector("#monBouton");
const titre = document.querySelector("h1");

// 2. Écouter un événement (clic, survol...)
bouton.addEventListener("click", () => {
    // 3. Modifier le contenu ou le CSS
    titre.textContent = "Nouveau Titre !";
    titre.style.color = "#38bdf8";
});
```

3.4 Java & Programmation Orientée Objet (POO)

Les 4 Piliers de la POO en Java :

- **Encapsulation** : Protection des données via des attributs `private` et accès par getters/setters.
- **Héritage** (`extends`) : Une classe fille hérite des propriétés et méthodes d'une classe mère.
- **Polymorphisme** : Capacité d'une méthode à prendre plusieurs formes (surcharge/redéfinition).

- **Abstraction** : Utilisation de classes abstraites ou d'`interface` pour définir des contrats de code.

```
// Exemple de Classe Java de base
public class Etudiant {
    // Attributs privés (Encapsulation)
    private String nom;
    private int age;

    // Constructeur
    public Etudiant(String nom, int age) {
        this.nom = nom;
        this.age = age;
    }

    // Getter
    public String getNom() {
        return this.nom;
    }

    // Méthode principale d'exécution
    public static void main(String[] args) {
        Etudiant e1 = new Etudiant("Alexis", 19);
        System.out.println("Nom de l'étudiant : " + e1.getNom());
    }
}
```